Supplementary Material: Detailed BAN Logic and ProVerif Verification for Blockchain-based Cross-layer Authentication Scheme for 5G Networks

Chenyu Wang, Dinghao Wang, Xiaofeng Tao, Shaoyong Guo, and Qimei Cui

Beijing University of Posts and Telecommunications, Beijing 100876, China
wangchenyu@bupt.edu.cn

This supplementary document provides the detailed ProVerif model and BAN-logic derivations omitted from the conference paper because of space limitations. The presentation follows the same WASA/LNCS formatting style as the main manuscript so that the supplementary proof can be read as a natural extension of the paper.

1 Detailed ProVerif Verification

To present the symbolic verification first, this section defines the communication channels and cryptographic primitives, then encodes the UE, HN, and SN behaviors, and finally issues secrecy and correspondence queries. This organization makes it possible to check both confidentiality and authentication in the standard symbolic-adversary model. The modeling strategy follows the standard Dolev–Yao abstraction: cryptographic primitives are treated as ideal symbolic operators, while the adversary is allowed to intercept, replay, and synthesize messages on public channels. This abstraction is well suited to the present protocol because the main goal is to confirm that the message flow, key dependencies, and event correspondences are structurally sufficient to preserve secrecy and mutual authentication.

In the ProVerif model, the UE–SN link is intentionally left public to reflect the exposure of the wireless side, whereas the SN–HN link is modeled as private in accordance with the trust assumptions adopted in the main paper. The code fragments reproduced below are not intended to exhaust every implementation detail; instead, they retain the essential computations that determine whether the authentication run can be completed and whether the derived session key material remains protected.

1.1 Premises and Security Queries

The main ProVerif premises used in the model are shown below.

Listing 1.1: Core ProVerif premises and secrecy queries.

```
free UE_SN: channel.
free SN_HN: channel [private].

type skey.
fun pk(skey): pkey.
fun aenc(bitstring, pkey): bitstring.
reduc forall m: bitstring, sk: skey;
      adec(aenc(m, pk(sk)), sk) = m.

fun challenge(key, nonce, SNID): xres_star.
fun keyseed(key, nonce, SQN, SNID): key_seaf.
fun f1(key, bitstring): MAC.
fun f5(key, nonce): AK.

query attacker(k_1);
      attacker(k_2);
      attacker(sk_hn);
      attacker(supi_1);
      attacker(supi_2).
query secret sqn_ue;
      secret sqn_hn;
      secret k_seaf.
```

These declarations capture the insecure UE–SN channel, the private SN–HN channel, the asymmetric encryption/decryption used by ECIES, the challenge/key-derivation functions, and the secrecy goals on $SUPI$, SQN, and k_{seaf}. In particular, the secrecy queries are aligned with the main security claims of the paper: they test whether the symbolic attacker can derive the long-term key material, user identity, sequence numbers, or the final access-session key. The model therefore checks not only confidentiality of static data but also the confidentiality of the transient values that drive successful 5G authentication.

1.2 Modeled Principal Processes

The three principal processes below mirror the functional split used in the main protocol description. The UE protects the subscriber identity, verifies the challenge, and generates the response; the HN rebuilds the subscriber state and prepares the challenge vector; and the SN mainly forwards messages while recording the authentication events used later in the correspondence queries. This organization keeps the symbolic model close to the real protocol logic and makes the role of each principal easier to follow.

Listing 1.2: UE process used in the ProVerif model.

```
let processUE(k: key, supi_ue: SUPI, pkHN: pkey, idHN: hostid) =
  new sqn: SQN;
  new Rs: nonce;
  in(UE_SN, SNname: SNID);
  let suci_ue = encapsulate_to_suci(aenc((supi_ue, Rs, sqn), pkHN), idHN) in
  out(UE_SN, suci_ue);
  in(UE_SN, m4: bitstring);
  let (rand: nonce, autn: AUTN) = m4 in
  let encapsulate_to_autn(xconc, xmac) = autn in
  let ak: AK = f5(k, rand) in
  let xsqn: SQN = dxor(xconc, ak) in
  let xmac: MAC = f1(k, (xsqn, rand, SNname)) in
  if mac = xmac then
    if sqn = xsqn then
      let res_star: xres_star = challenge(k, rand, SNname) in
      let k_seaf: key_seaf = keyseed(k, rand, xsqn, SNname) in
      out(UE_SN, res_star).
```

The UE process above highlights the two essential verification points on the user side: the symbolic counterpart of MAC checking and the confirmation that the sequence-number relation is valid. Only when both conditions hold does the UE produce res^* and the session key seed, which is consistent with the challenge–response logic of the protocol.

Listing 1.3: HN process used in the ProVerif model.

```
let processHN(skHN: skey, idHN: hostid) =
  in(SN_HN, m2: bitstring);
  let (suci: SUCI, SNname: SNID) = m2 in
  if id = idHN then
    let (supi_hn: SUPI, Rs: nonce, sqn: SQN) = adec(x, skHN) in
    get hss(=supi_hn, k_hn: key) in
    new rand: nonce;
    let mac: MAC = f1(k_hn, (sqn, rand, SNname)) in
    let ak: AK = f5(k_hn, rand) in
    let xres: xres_star = challenge(k_hn, rand, SNname) in
    let hxres: bitstring = sha256((rand, xres)) in
    let k_seaf: key_seaf = keyseed(k_hn, rand, sqn, SNname) in
    out(SN_HN, (rand, autn, hxres, k_seaf));
    in(SN_HN, m6: bitstring);
    if res = xres then out(SN_HN, supi_hn).
```

At the HN side, the model reconstructs the concealed subscription information, generates fresh challenge material, and derives the values that later drive authentication on both the UE and SN sides. This symbolic process is important because it binds the user identity, the challenge, and k_{seaf} to the same protocol run.

```
Query not attacker_key(k_1[]) is true.
Query not attacker_key(k_2[]) is true.
Query not attacker_skey(sk_hn[]) is true.
Query not attacker_SUPI(supi_1[]) is true.
Query not attacker_SUPI(supi_2[]) is true.
Query secret sqn_ue_1,sqn_ue is true.
Query secret sqn_hn is true.
Query secret k_seaf_3,k_seaf_2,k_seaf_1,k_seaf is true.
Query event(snEndWithUe(ue,sn)) ==> event(ueAuthSn(ue,sn)) is true.
Query event(ueEndWithHn(ue,hn)) ==> event(hnAuthUe(ue,hn)) is true.
Query event(ueEndWithHnOnSNID(ue,hn,sn)) ==> event(hn2ueOnSNID(ue,hn,sn)) is true.
Query event(hnEndWithUeOnSNID(ue,hn,sn)) ==> event(ue2hnOnSNID(ue,hn,sn)) is true.
Query event(snEndWithHnOnSUPI(sn,ue)) ==> event(hn2snOnSUPI(sn,ue)) is true.
Query event(hnEndWithSnOnSUCI(sn,ue)) ==> event(sn2hnOnSUCI(sn,ue)) is true.
Query event(ueEndWithHnOnKseaf(ue,hn,k_seaf_4)) ==> event(hn2ueOnKseaf(ue,hn,k_seaf_4)) is true.
Query inj-event(ueEndWithHnOnKseaf(ue,hn,k_seaf_4)) ==> inj-event(hn2ueOnKseaf(ue,hn,k_seaf_4)) is true.
Query inj-event(hnEndWithUeOnKseaf(ue,hn,k_seaf_4)) ==> inj-event(ue2hnOnKseaf(ue,hn,k_seaf_4)) is true.
Query inj-event(hnEndWithSnOnKseaf(sn,k_seaf_4)) ==> inj-event(sn2hnOnKseaf(sn,k_seaf_4)) is true.
```

Fig. 1: Representative ProVerif verification result reproduced in the supplementary material.

Listing 1.4: SN process and representative authentication events.

```
let processSN(SNname: SNID) =
  out(UE_SN, SNname);
  in(UE_SN, suci: SUCI);
  out(SN_HN, (suci, SNname));
  in(SN_HN, m3: bitstring);
  let (rand: nonce, autn: AUTN, hxres: bitstring, k_seaf: key_seaf) = m3 in
  event snAuthUe(suci, SNname);
  event sn2hnOnSUCI(SNname, suci);
  event sn2hnOnKseaf(SNname, k_seaf);
  event sn2ueOnKseaf(suci, SNname, k_seaf);
  out(UE_SN, (rand, autn));
  in(UE_SN, res_star: xres_star);
  if hxres = sha256((rand, res_star)) then
    out(SN_HN, (res_star, suci));
    in(SN_HN, supi_sn: SUPI);
    event snEndWithUe(supi_sn, SNname);
    event snEndWithHnOnSUPI(SNname, supi_sn);
    event snEndWithUeOnKseaf(supi_sn, SNname, k_seaf);
    event snEndWithHnOnKseaf(SNname, k_seaf).
```

The SN process mainly preserves the protocol ordering and records representative events. These events are the anchor points of the later correspondence queries, so they are crucial for showing that a completed run on one side is matched by a genuine run on the other side, rather than by an adversarially fabricated transcript.

1.3 Verification Results and Interpretation

Representative query outcomes are summarized by the result figure reproduced below. The figure is shown only as a compact representative output; the accompanying interpretation is provided in ordinary text so that the reader can see more clearly how the reported 'true' results map to the secrecy and correspondence goals discussed in the paper.

The detailed output confirms that the attacker cannot derive k_1, k_2, sk_{hn}, $SUPI$, sqn_{ue}, sqn_{hn}, or k_{seaf}. It also validates the expected event correspondences, including the injective correspondences on k_{seaf} that bind the UE, HN,

and SN views of the same authenticated session. These results jointly support both secrecy and mutual authentication under the Dolev–Yao model.

More specifically, the secrecy queries show that the symbolic attacker never gains access to the protected subscriber identity, sequence numbers, long-term key material, or the session key seed even though the wireless-side channel is public. The correspondence queries then complement this secrecy evidence by showing that the endpoint events observed by the UE, HN, and SN cannot be completed in isolation: they are linked to one another through the event structure of the protocol run.

For completeness, the representative correspondence queries checked in the detailed appendix version include:

Listing 1.5: Representative correspondence queries.

```
query event(snEndWithUe(ue,sn)) ==> event(ueAuthSn(ue,sn)).
query event(ueEndWithHn(ue,hn)) ==> event(hnAuthUe(ue,hn)).
query event(snEndWithHnOnSUPI(sn,ue)) ==> event(hn2snOnSUPI(sn,ue)).
query inj-event(ueEndWithHnOnKseaf(ue,hn,k_seaf_4))
   ==> inj-event(hn2ueOnKseaf(ue,hn,k_seaf_4)).
query inj-event(hnEndWithUeOnKseaf(ue,hn,k_seaf_4))
   ==> inj-event(ue2hnOnKseaf(ue,hn,k_seaf_4)).
query inj-event(hnEndWithSnOnKseaf(sn,k_seaf_4))
   ==> inj-event(sn2hnOnKseaf(sn,k_seaf_4)).
```

The above queries cover both ordinary and injective correspondences. Ordinary correspondences confirm that one party's completion event implies that the peer has previously executed the matching step, while injective correspondences additionally prevent one valid run from being reused to justify multiple distinct sessions. In the context of 5G authentication, this distinction is important because it strengthens the interpretation of the symbolic result from simple reachability to authentication consistency across different runs.

2 Detailed BAN Logic Verification

Following the longer appendix version of the manuscript, the BAN-logic verification is organized into four parts: notation/rules, idealized messages, assumptions/goals, and derivations. The purpose is to show step by step how the protocol establishes the intended beliefs of the HN, UE, and SN during the initialization and challenge-response phases.

2.1 Initialization Stage

The idealized message of the initialization phase is

$$UE \rightarrow HN : \langle SUPI, UE \overset{k_2}{\Leftrightarrow} HN \rangle_{k_1}. \tag{A1}$$

The BAN-logic assumptions adopted for this stage are

$$HN \mid\equiv UE \overset{k_1}{\Leftrightarrow} HN, \tag{A2}$$

$$HN \mid\equiv \#((UE \overset{k_1}{\Leftrightarrow} HN)), \tag{A3}$$

Table 1: Core notations and rules used in the BAN-logic derivation.

Notation	**Meaning**	**Rule**	**Formula**
$P \mid\equiv X$	P believes X	Message Meaning (MM)	$P \mid\equiv P \leftrightarrow_K Q,\ P \lhd \{X\}_K \Rightarrow P \mid\equiv Q \mid\sim X$
$P \lhd X$	P sees X	Nonce Verification (NV)	$P \mid\equiv \#(X),\ P \mid\equiv Q \mid\sim X \Rightarrow P \mid\equiv Q \mid\equiv X$
$P \mid\sim X$	P once said X	Jurisdiction (JR)	$P \mid\equiv Q \Rightarrow X,\ P \mid\equiv Q \mid\equiv X \Rightarrow P \mid\equiv X$
$P \Rightarrow X$	P has authority over X	Freshness (FR)	$P \mid\equiv \#(X) \Rightarrow P \mid\equiv \#((X,Y))$
$\#(X)$	X is fresh	Decomposition (DR)	$P \lhd (X,Y) \Rightarrow P \lhd X$
$\langle X\rangle_K$	X combined with secret K	Belief Conjunction (BC)	$P \mid\equiv X,\ P \mid\equiv Y \Rightarrow P \mid\equiv (X,Y)$
$\{X\}_K$	X encrypted by K	Hash Rule (HR)	$P \mid\equiv Q \mid\sim H(X),\ P \lhd X \Rightarrow P \mid\equiv Q \mid\sim X$

$$HN \mid\equiv UE \overset{k_2}{\Leftrightarrow} HN, \tag{A4}$$

$$HN \mid\equiv \#((UE \overset{k_2}{\Leftrightarrow} HN)). \tag{A5}$$

The goal is that the HN should eventually believe that the UE itself believes the recovered identity:

$$HN \mid\equiv UE \mid\equiv SUPI. \tag{A6}$$

The derivation proceeds as follows.

$$HN \lhd \left\langle \{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, UE \overset{k_1}{\Leftrightarrow} HN \right\rangle_{k_1}, \tag{A7}$$

$$HN \mid\equiv UE \mid\sim \left(\{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, UE \overset{k_1}{\Leftrightarrow} HN\right), \tag{A8}$$

$$HN \mid\equiv UE \mid\equiv \{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, \tag{A9}$$

$$HN \mid\equiv UE \mid\equiv SUPI. \tag{A10}$$

Equation (A7) follows from the received initialization message. Using the message-meaning rule with (A2) yields (A8); freshness and nonce verification with (A3) then give (A9). Finally, by combining (A9) with the belief that k_2 is a valid UE–HN secret and fresh, as assumed in (A4)–(A5), the HN concludes (A10), namely that the received $SUPI$ is endorsed by the UE.

2.2 Challenge-response Stage

The idealized messages of the challenge-response phase are written as

$$HN \rightarrow UE : RAND',\ \{SQN_{HN}\}_{AK},\ \left\langle SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} HN \right\rangle_{k_1}, \tag{A11}$$

$$UE \rightarrow SN : res^*,\ hres^*, \tag{A12}$$

$$UE \rightarrow HN : \left\langle ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF \right\rangle_k, \tag{A13}$$

$$HN \rightarrow SN : \left\{ SUPI, UE \overset{k_{seaf}}{\leftrightarrow} SEAF, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF) \right\}_{K_{N12}}. \tag{A14}$$

The assumptions used in this phase are

$$UE \mid\equiv UE \overset{k}{\leftrightarrow} HN, \tag{A15}$$
$$UE \mid\equiv X \Rightarrow UE, \tag{A16}$$
$$UE \mid\equiv \#(SQN_{HN}), \tag{A17}$$
$$UE \mid\equiv \#(k_{seaf}), \tag{A18}$$
$$SN \mid\equiv RAND', \tag{A19}$$
$$SN \mid\equiv hres^*, \tag{A20}$$
$$HN \mid\equiv UE \overset{k}{\Leftrightarrow} HN, \tag{A21}$$
$$HN \mid\equiv \#(RAND'), \tag{A22}$$
$$SN \mid\equiv SN \overset{K_{N12}}{\leftrightarrow} HN, \tag{A23}$$
$$HN \mid\equiv \#((SN \overset{K_{N12}}{\leftrightarrow} HN)), \tag{A24}$$
$$SN \mid\equiv HN \Rightarrow (UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A25}$$
$$SN \mid\equiv HN \Rightarrow (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF). \tag{A26}$$

Because k_{seaf} is regenerated from fresh ECIES/ECDH material and the SN receives $RAND'$ and $HXRES^*$ through the secure SN–HN channel, the following intermediate goals are justified:

$$UE \mid\equiv HN \mid\equiv RAND', \tag{A27}$$
$$UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A28}$$
$$UE \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A29}$$
$$HN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A30}$$
$$SN \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A31}$$
$$SN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF. \tag{A32}$$

The detailed derivation is

$$UE \lhd RAND', \tag{A33}$$
$$UE \mid\equiv UE \overset{AK}{\leftrightarrow} HN, \tag{A34}$$
$$UE \lhd \{SQN_{HN}\}_{AK}, \tag{A35}$$
$$UE \mid\equiv HN \mid\equiv SQN_{HN}, \tag{A36}$$
$$UE \lhd \left\langle SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF \right\rangle_k, \tag{A37}$$
$$UE \mid\equiv HN \mid\sim (SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A38}$$

$$UE \mid\equiv HN \mid\equiv (SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A39}$$
$$UE \mid\equiv HN \mid\equiv RAND', \tag{A40}$$
$$UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A41}$$
$$UE \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A42}$$
$$SN \lhd res^*, \tag{A43}$$
$$SN \mid\equiv res^*, \tag{A44}$$
$$HN \lhd \left\langle ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} HN \right\rangle_k, \tag{A45}$$
$$HN \mid\equiv UE \mid\equiv (ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A46}$$
$$HN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A47}$$
$$SN \lhd \left\{ SUPI, UE \overset{k_{seaf}}{\leftrightarrow} HN, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF) \right\}_{K_{N12}}, \tag{A48}$$
$$SN \mid\equiv HN \mid\equiv (SUPI, UE \overset{k_{seaf}}{\leftrightarrow} HN, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF)), \tag{A49}$$
$$SN \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A50}$$
$$SN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A51}$$
$$SN \mid\equiv HN \mid\equiv (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A52}$$
$$SN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF. \tag{A53}$$

Equations (A33)–(A40) show that the UE first authenticates the freshness of $RAND'$ and then accepts the HN's belief about the current k_{seaf} binding. Equations (A41)–(A47) capture the HN-side confirmation that the UE has accepted the same session-key context. Finally, Eqs. (A48)–(A53) show how the SN, using the protected K_{N12} message and the jurisdiction assumptions in (A25)–(A26), concludes both that $UE \overset{k_{seaf}}{\leftrightarrow} SEAF$ holds and that the UE itself believes this binding. Hence, the challenge-response phase establishes mutual authentication together with a shared belief in the freshness and legitimacy of k_{seaf}.

3 Closing Remark

This supplementary document retains the detailed ProVerif and BAN-logic materials omitted from the conference version. Read together with the main paper, it provides a complete presentation of the protocol's symbolic-verification evidence and belief-level reasoning.